IaC Provisioning for Real Estate System

Course Name:

DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	CHERRY MEHTA	EN22CS301292
2	BHUMIKA CHOYAL	EN22CS301276
3	AYUSH JAIN	EN22CS301247
4	AYUSH PATIDAR	EN22CS301253
5	DIYA GOYAL	EN22CS301351

Group Name: 03D3

Project Number: DO-27

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2026

Table of Contents

Introduction

1. Scope of the Document
2. Intended Audience
3. System Overview

System Design

1. Application Design
2. Process Flow
3. Information Flow
4. Component Design
5. Key Design Consideration
6. API Catalogue

Data Design

1. Data Model
2. Data Access Mechanism
3. Data Retention Policies
4. Data Migration

Interfaces

State and Session Management

Caching

Non Functional Requirements

1. Security Aspects
2. Performance Aspects

References

Introduction

This document presents the High-Level Design (HLD) of the IaC Provisioning for Real Estate System. The primary goal of this project is to automate the provisioning, configuration, and deployment of infrastructure required to host a Real Estate application using DevOps practices.
Traditional infrastructure setup involves manual configuration, which is time-consuming and error-prone. This project leverages Infrastructure as Code (IaC) tools such as Terraform and Ansible to automate infrastructure creation and configuration. Additionally, containerization using Docker and CI/CD automation using Jenkins or GitHub Actions ensures seamless deployment.
The system supports both cloud-based deployment (AWS) and local containerized environments, making it flexible for development and production usage.

1. Scope of the Document

This document includes:

- Design of automated infrastructure provisioning
- Deployment strategy using Docker containers
- Integration of CI/CD pipelines
- Cloud infrastructure setup using AWS
- Configuration management using Ansible
- Security, scalability, and performance considerations
- High-level workflows and architecture

This document does not include:

- Detailed source code implementation
- UI/UX design of the Real Estate application
- Business logic of the application

2. Intended Audience

This document is intended for:

- DevOps Engineers – to understand automation workflows and infrastructure design
- Developers – to understand deployment pipelines and environment setup
- System Administrators – to manage deployed infrastructure
- Project Evaluators – to assess system design and architecture

- Cloud Engineers – to understand provisioning strategies

Basic familiarity with Docker, AWS, and CI/CD is assumed.

3. System Overview

The system automates the deployment lifecycle of a Real Estate application. It integrates multiple DevOps tools to ensure that infrastructure provisioning, configuration, and deployment are executed automatically.

Key functionalities include:

- Provisioning cloud infrastructure (EC2, VPC, IAM, S3)
- Configuring servers using Ansible
- Containerizing applications using Docker
- Automating deployments using CI/CD pipelines
- Monitoring deployed services

Key benefits:

- Eliminates manual errors
- Ensures consistency across environments
- Enables rapid deployment
- Supports scalability and high availability

System Design

1. Application Design

The system follows a layered and modular architecture, where each layer performs a specific role independently but integrates seamlessly.

Layers Explained:

Infrastructure Layer (Terraform):
Responsible for provisioning cloud resources like EC2 instances, networking components, and storage.

Configuration Layer (Ansible):
Installs required software, dependencies, and configures the servers after provisioning.

Container Layer (Docker):
Packages the application into containers ensuring portability and consistency.

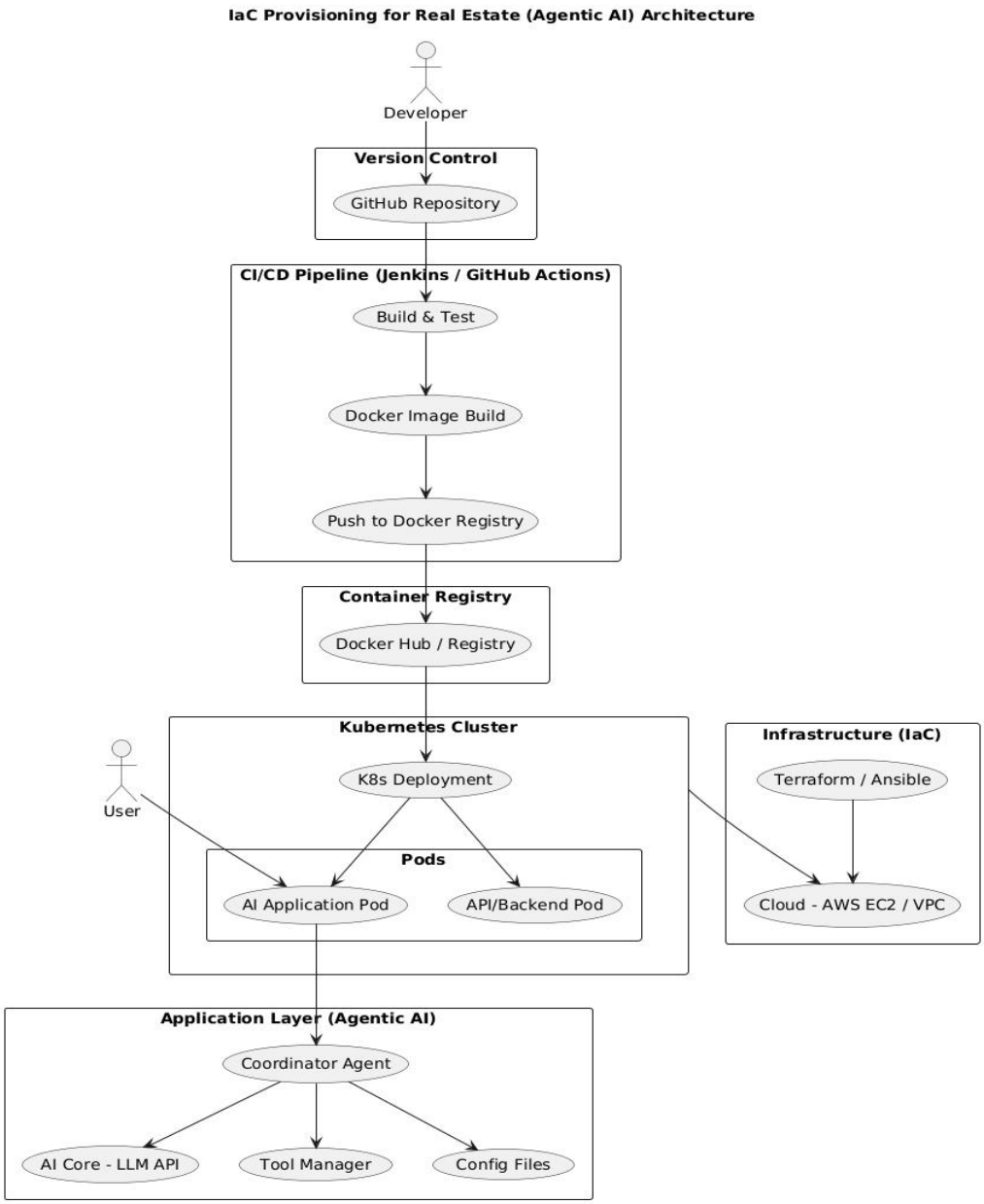
CI/CD Layer (Jenkins/GitHub Actions):
Automates build, test, and deployment processes.

Version Control Layer (Git):
Maintains source code, infrastructure code, and configuration scripts.

Execution Layer (AWS/Local):
The environment where the application is deployed and executed.

Design Approach:

- Modular Terraform modules for reusability
- Ansible roles for structured configuration
- Stateless container-based deployments
- Separation of concerns between infrastructure and application



2. Process Flow

The workflow ensures complete automation from code commit to deployment.

Detailed Flow:

- Developer pushes code to Git repository
- CI/CD pipeline is triggered automatically
- Pipeline validates code and builds Docker image
- Terraform initializes and provisions infrastructure
- Infrastructure (EC2, networking) is created on AWS
- Ansible connects to instances via SSH
- Required dependencies (Docker, runtime) are installed
- Docker containers are deployed on provisioned instances
- Application becomes accessible to end users
- Logs and monitoring tools track system health

3. Information Flow

The system ensures smooth data and control flow between components:

- Source Code Flow: Developer → Git → CI/CD
- Infrastructure Instructions: Terraform → AWS APIs
- Configuration Flow: Ansible → EC2 Instances
- Application Flow: Docker → Running Containers
- Monitoring Flow: Logs → Monitoring tools

This structured flow ensures traceability and automation.

4. Component Design

Git Repository

- Stores application code, Terraform scripts, and Ansible playbooks
- Enables version control and collaboration

Terraform

- Defines infrastructure in code (HCL)
- Creates AWS resources like EC2, VPC, S3

Ansible

- Automates configuration and software installation
- Ensures idempotent execution

Docker

- Packages application and dependencies
- Ensures environment consistency

CI/CD Tool (Jenkins/GitHub Actions)

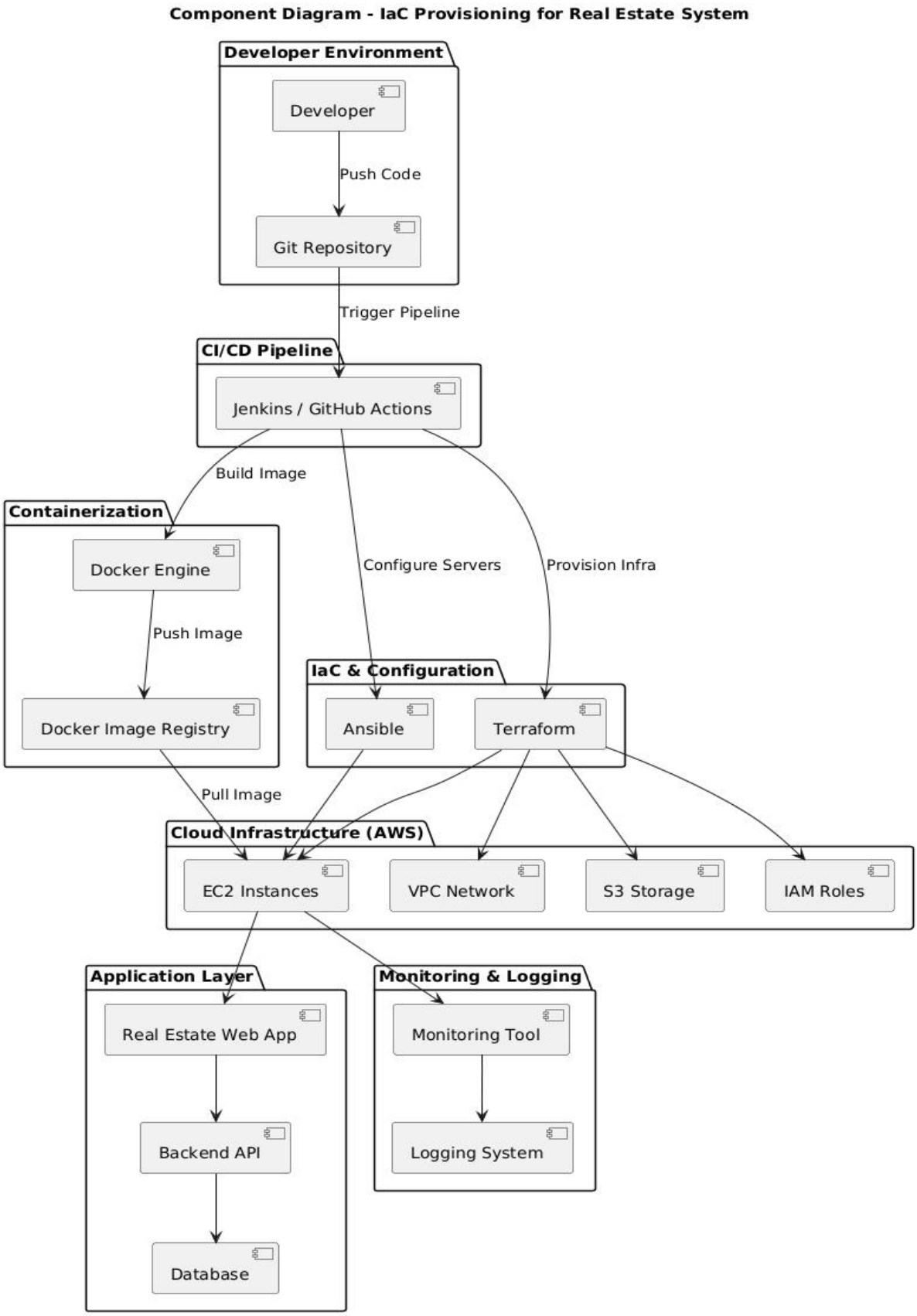
- Automates build and deployment
- Integrates all components

AWS Cloud

- Hosts infrastructure
- Provides scalability and reliability

Monitoring & Logging

- Tracks performance and failures
- Enables alerting mechanisms


Component Diagram - IaC Provisioning for Real Estate System
Developer Environment
Developer
Push Code
Git Repository
Trigger Pipeline
CI/CD Pipeline
Jenkins / GitHub Actions
Build Image
Configure Servers
Provision Infra
Containerization
Docker Engine
Push Image
Docker Image Registry
IaC & Configuration
Ansible
Terraform
Pull Image
Cloud Infrastructure (AWS)
EC2 Instances
VPC Network
S3 Storage
IAM Roles
Application Layer
Real Estate Web App
Backend API
Database
Monitoring & Logging
Monitoring Tool
Logging System

5. **Key Design Considerations**

- Scalability: Ability to handle increased user load using auto-scaling
- Security: Use of IAM roles, encrypted storage, secure communication
- High Availability: Use of multiple availability zones
- Modularity: Independent and reusable components
- Fault Tolerance: System recovery mechanisms
- Cost Efficiency: Use of optimized cloud resources
- Automation: Minimal manual intervention
- Consistency: Same setup across all environments

6. **API Catalogue**

S.No	API Name	Purpose	Method	Description
1	AWS EC2 API	Instance Creation	POST	Launch virtual machines
2	AWS VPC API	Network Setup	POST	Configure networking
3	AWS S3 API	Storage	PUT/GET	Store data and state files
4	Docker API	Container Mgmt	REST	Manage containers
5	Terraform CLI	Infra Execution	CLI	Apply infrastructure code
6	Ansible SSH	Config Mgmt	SSH	Configure instances
7	Jenkins API	Pipeline Trigger	POST	Automate workflows

Data Design

1. Data Model

The system handles both infrastructure and application-level data.

Types of Data:

- Terraform state files
- Configuration files (YAML, JSON, HCL)
- Application data (property listings, users)
- Logs and monitoring data

Entities:

- Infrastructure resources

- Deployment configurations
- User data
- Logs

2. Data Access Mechanism

- Secure SSH-based communication
- IAM-based authentication
- Role-based access control
- Encrypted storage using AWS S3
- API-based communication

3. Data Retention Policies

- Terraform state stored with versioning in S3
- Logs retained for audit (30–90 days)
- Backup strategies (daily/weekly)
- Sensitive data encrypted

3. Data Migration

- Automated migration scripts
- Backup and restore mechanisms
- Version-controlled schema updates
- Zero downtime deployments

Interfaces

The system supports multiple interfaces:

- CLI (Terraform, Ansible)
- Cloud Console (AWS Management Console)

- Web Interface (Real Estate Application)
- REST APIs
- Monitoring Dashboards

State and Session Management

- Terraform remote backend using S3 with state locking
- Stateless application containers
- Session management using database or Redis
- IAM-based session handling

Caching

- Redis for application-level caching
- Docker layer caching for faster builds
- Terraform plugin caching
- CDN caching for faster content delivery

Non-Functional Requirements

1. **Security Aspects**
    - IAM role-based access control
    - Encrypted data storage (S3 encryption)
    - HTTPS secure communication
    - SSH key-based authentication
    - Secrets management (API keys, credentials)
    - Firewall and security groups

2. **Performance Aspects**
    - Auto-scaling groups for handling load

- Load balancers for traffic distribution
- Optimized Docker images
- Parallel infrastructure provisioning
- Real-time monitoring and alerts

References

- HashiCorp, Terraform Documentation. Available: https://developer.hashicorp.com/terraform/docs
- Red Hat, Ansible Documentation. Available: https://docs.ansible.com/
- Docker, Inc., Docker Documentation. Available: https://docs.docker.com/
- Amazon Web Services, AWS Documentation. Available: https://docs.aws.amazon.com/
- Jenkins Project, Jenkins Documentation. Available: https://www.jenkins.io/doc/